

Intro to Javascript

BASICS OF JS

JavaScript (often abbreviated as JS) is a versatile, high-level programming language primarily used to create interactive and dynamic content on websites. While HTML structures a webpage and CSS styles it, JavaScript brings it to life by allowing developers to manipulate content, respond to user actions, and communicate with servers.

What is Javascript?

2

□ a lightweight programming language ("scripting language")

- Make buttons click and things animate
- Validate forms (check if emails are correct)
- Update content without reloading the page
- Handle user actions (clicks, typing, scrolling)
- Connect to servers to fetch data

Key Features

- Easy to start learning
- Runs everywhere (web, mobile apps, servers)
- Fast execution in browsers
- Lots of free libraries and tools available

What is Javascript?

3

- a web standard (but not supported identically by all browsers)
- NOT related to Java other than by name and some syntactic similarities

Linking to a JavaScript file:

script

4

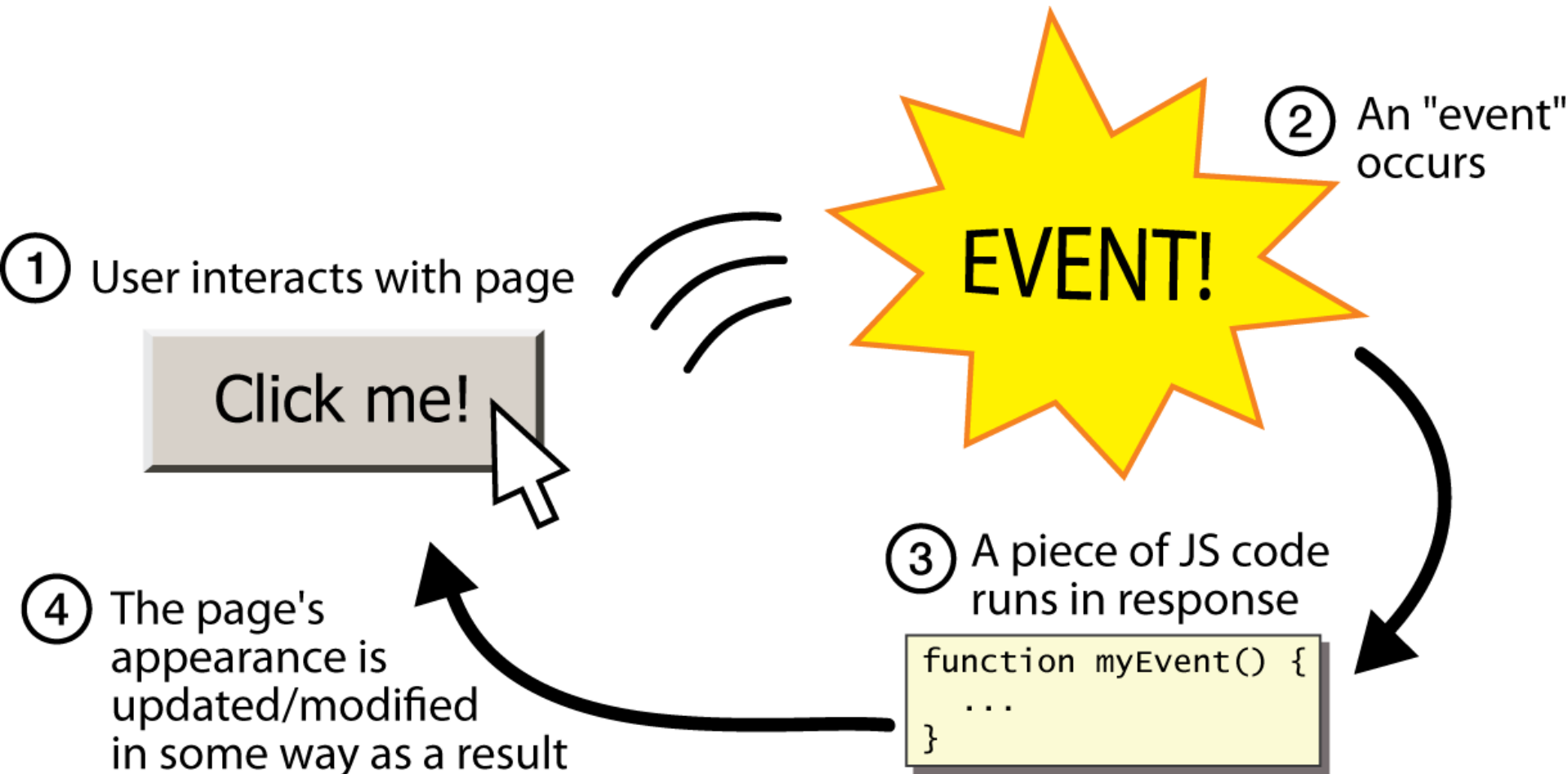
```
<script src="filename" type="text/javascript"></script>
```

HTML

- script tag should be placed in HTML page's head
- script code is stored in a separate .js file
- JS code can be placed directly in the HTML file's body or head (like CSS)
 - but this is bad style (should separate content, presentation, and behavior)

Event-driven programming

5



JavaScript Versions

6

the different versions of ECMAScript, their release years, and the key features they introduced:

Version	Year	Key Features
ES5	2009	strict mode, JSON, getters/setters
ES6	2015	let/const, classes, arrow functions
ES7-ES13	2016-2022	async/await, BigInt, optional chaining
ES14	2023	toSorted, findLast, static blocks

JavaScript Variables

7

- ❑ Variables in JavaScript are used to store data values. They can be declared in different ways depending on how the value should behave.
- ❑ Variables can be declared using `var`, `let`, or `const`.
- ❑ JavaScript is dynamically typed, so types are decided at runtime.
- ❑ You don't need to specify a data type when creating a variable.

Rules for Naming Variables

8

- ❑ `let userName = "Suman"; // Valid`
- ❑ `let $price = 100; // Valid`
- ❑ `let _temp = 0; // Valid`
- ❑ `let 123name = "Ajay"; // Invalid`
- ❑ `let function = "mpec"; // Invalid`

How to use?

9

// Old style

```
var a = 10
```

// Preferred for non-const

```
let b = 20;
```

// Preferred for const (cannot be changed)

```
const c = 30;
```

```
console.log(a);
```

```
console.log(b);
```

```
console.log(c);
```

Declaring Variables in JavaScript

10

- Variables were declared only with var, which is function-scoped and global-scoped, causing issues like hoisting and global pollution.
- Scope: let and const are block-scoped (limited to { } block) or global-scoped, reducing errors compared to var.
- **1. var keyword**
- var is a keyword in JavaScript used to declare variables and it is Function-scoped and hoisted, allowing redeclaration but can lead to unexpected bugs.

2. let keyword

let is a keyword in JavaScript used to declare variables and it is Block-scoped and not hoisted to the top, suitable for mutable variables

```
let a = 12  
let b = "mpec"; console.log(a);  
  
console.log(b);
```

3. const keyword

const is a keyword in JavaScript used to declare variables and it is Block-scoped, immutable bindings that can't be reassigned, though objects can still be mutated.

```
const a = 5  
let b = "mpec"; console.log(a);  
console.log(b);
```

Scope of Variables in JavaScript

12

- Scope determines where a variable can be accessed or used within a JavaScript program. It helps control the visibility and lifetime of variables in different parts of the code.

Global Scope

A global variable refers to a variable that is declared outside any function or block, so it can be used anywhere in the program, both inside functions and in the main code.

13

```
// Global Variable accessed from within a function
```

```
const x = 10;
```

```
function fun1() {
```

```
    console.log(x);
```

```
}
```

```
fun1();
```

Local Scope

A local variable is a variable declared inside a function, making it accessible only within that function. It cannot be used outside the function.

```
function fun2(){
```

```
    // This variable is local to fun2() and
```

```
    // cannot be accessed outside this function
```

```
    let x = 10;
```

```
    console.log(x);
```

```
}
```

```
console.log(x);
```

```
fun2();
```

1. Global and Local Scope

14

```
// Declaring a global variable
let x = 10;
function func() {
  // Declaring a local variable
  let y = 20;
  // Accessing Local and Global
  // variables
  console.log(x, "", y);
}
func();
```

Block Scope

Block Scope refers to the area within conditional statements, loops, or any set of curly braces `{ }`. Variables declared inside a block cannot be accessed from outside that block...

15

```
{  
  let internalBox = "Secret";  
  var publicInfo = "I am visible everywhere!";  
}  
console.log(publicInfo); // Output: I am visible everywhere!  
console.log(internalBox); // Error: internalBox is not defined
```

Logic"What stays in the `{ }` stays in the `{ }`."

Lexical Scope

16

A **Lexical Scope** (also known as Static Scope) is the ability of a function to access variables from its parent scope based on where the function was **defined** in the source code. In JavaScript, an inner function always has access to the variables of its outer (parent) function, even after the parent function has finished executing.

```
function grandParent() {  
  let house = "Big Mansion";  
  function child() {  
    // The child can see the "house" because of Lexical Scope  
    console.log("I live in a " + house);  
  }  
  child();  
}  
grandParent(); // Output: I live in a Big Mansion
```

"I can see what my parents defined."

Cheatsheet

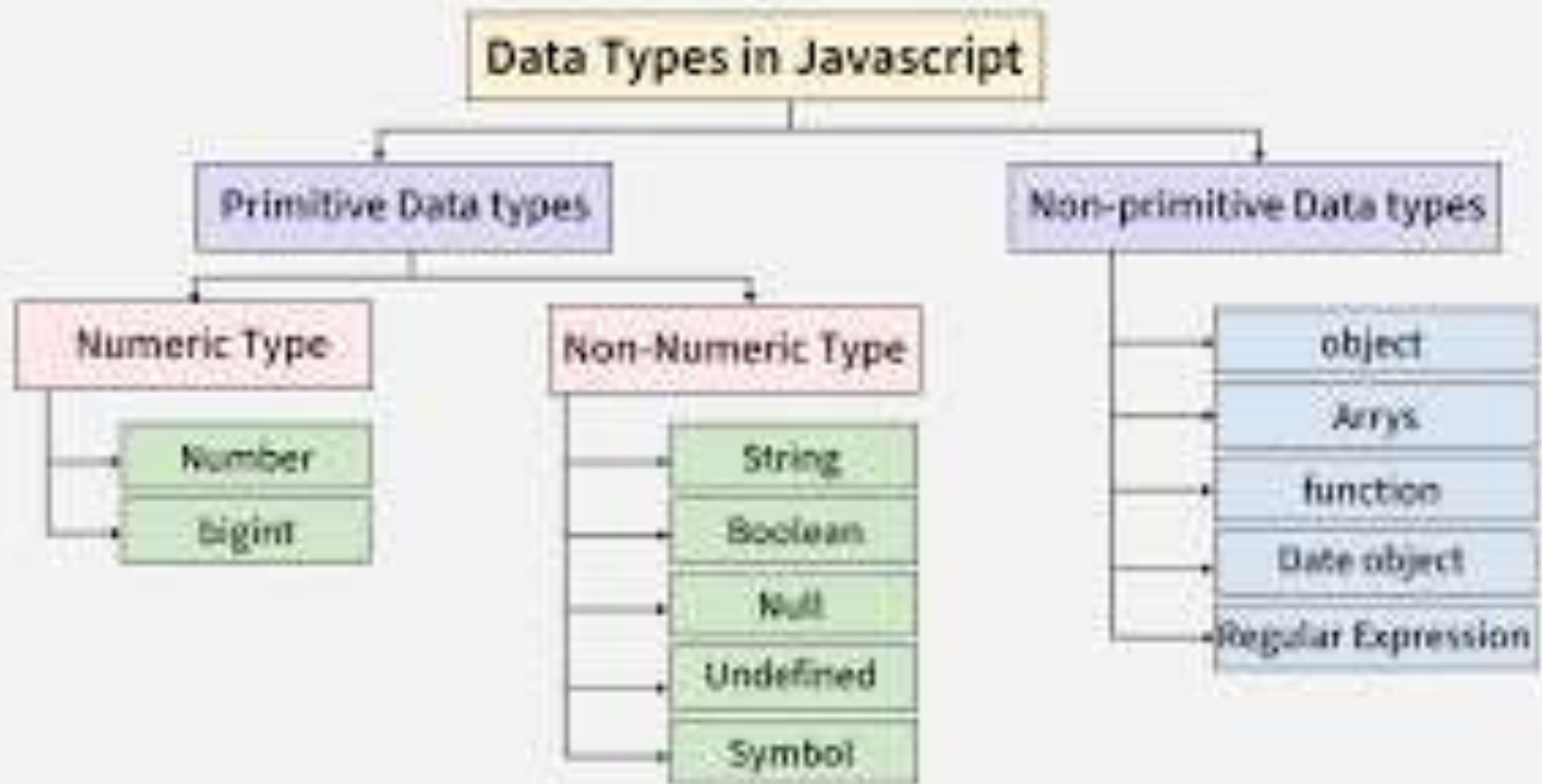
17

Feature	var	let	const
1. Global Scope	Accessible everywhere. Adds to window object.	Accessible everywhere. Does NOT add to window.	Accessible everywhere. Does NOT add to window.
2. Function Scope	Limited to the function.	Limited to the function.	Limited to the function.
3. Block Scope	No. It "leaks" out of { } (if/for/while).	Yes. Limited to the { } block.	Yes. Limited to the { } block.
4. Lexical Scope	Follows lexical rules (inner sees outer).	Follows lexical rules (inner sees outer).	Follows lexical rules (inner sees outer).
Redeclare?	Yes	No	No
Reassign?	Yes	Yes	No

Datatypes in JavaScript

18

Variables are used to store data values that can be used and modified in a program. JavaScript supports different data types to handle numbers, text, and other kinds of information.



Primitive Data types

19

Primitive datatypes represent single values and are immutable.

1. **Number**: Represents numeric values (integers and decimals).

```
let n = 42;
```

```
let pi = 3.14;
```

2. **String**: Represents text enclosed in single or double quotes.

```
let s = "Hello, World!";
```

3. **Boolean**: Represents a logical value (true or false).

```
let bool= true;
```

4. **Undefined**: A variable that has been declared but not assigned a value.

```
let notAssigned;
```

```
console.log(notAssigned);
```

5. **Null**: Represents an intentional absence of any value.

```
let empty = null;
```

6. **Symbol**: Represents unique and immutable values, often used as object keys.

```
let sym = Symbol('unique');
```

7. **BigInt**: Represents integers larger than `Number.MAX_SAFE_INTEGER`.

```
let bigNumber = 123456789012345678901234567890n;
```

```
let n1 = 2;  
console.log(n1)
```

```
let n2 = 1.3;  
console.log(n2)
```

```
let n3 = Infinity;  
console.log(n3)
```

```
let n4 = 'something here too' / 2;  
console.log(n4)
```

```
let s1 = "Hello There";  
console.log(s1);
```

```
let s2 = 'Single quotes work fine';  
console.log(s2);
```

```
let s3 = `can embed ${s1}`;  
console.log(s3);
```

```
let b1 = true;  
console.log(b1);
```

```
let b2 = false;  
console.log(b2);
```

```
let age = null;  
console.log(age)
```

```
let a;  
console.log(a);
```

Non-Primitive Datatypes

21

Non-primitive types are objects and can store collections of data or more complex entities.

1. **Object**: Represents key-value pairs.

```
let obj = {  
  name: "Amit",  
  age: 25  
};
```

2. **Array**: Represents an ordered list of values. Can be of any data type

```
let a = ["red", "green", "blue"];
```

3. **Function**: Represents reusable blocks of code. to perform a specific task when called.

```
function fun() {  
  console.log("JAVATUTORIALs");  
}
```

Objects

An object is a dynamic data structure that stores related data as key-value pairs, where each key uniquely identifies its value.

22

- The values of properties can be primitives, objects, or functions (known as methods when defined inside an object).
- Objects are mutable and dynamic properties can be added, modified, or deleted at any time.

□ 1. Creation Using Object Literal

The object literal syntax allows you to define and initialize an object with curly braces {}, setting properties as key-value pairs.

```
let obj = {  
  name: "Sourav",  
  age: 23,  
  job: "Developer."  
};  
console.log(obj);  
o/p  
{name: "Sourav", age: 23, job: "Developer."}
```

Creation Using new Object() Constructor

23

```
let obj = new Object();  
obj.name= "Sourav",  
obj.age= 23,  
obj.job= "Developer"  
console.log(obj);
```

Basic Operations on JavaScript Objects

Accessing Object Properties

```
let obj = { name: "Sourav", age: 23 };
```

```
// Using Dot Notation  
console.log(obj.name);
```

```
// Using Bracket Notation  
console.log(obj["age"]);
```

Modifying Object Properties

```
let obj = { name: "Sourav", age: 22 };  
console.log(obj);
```

```
obj.age = 23;  
obj.course="MCA";  
console.log(obj);
```

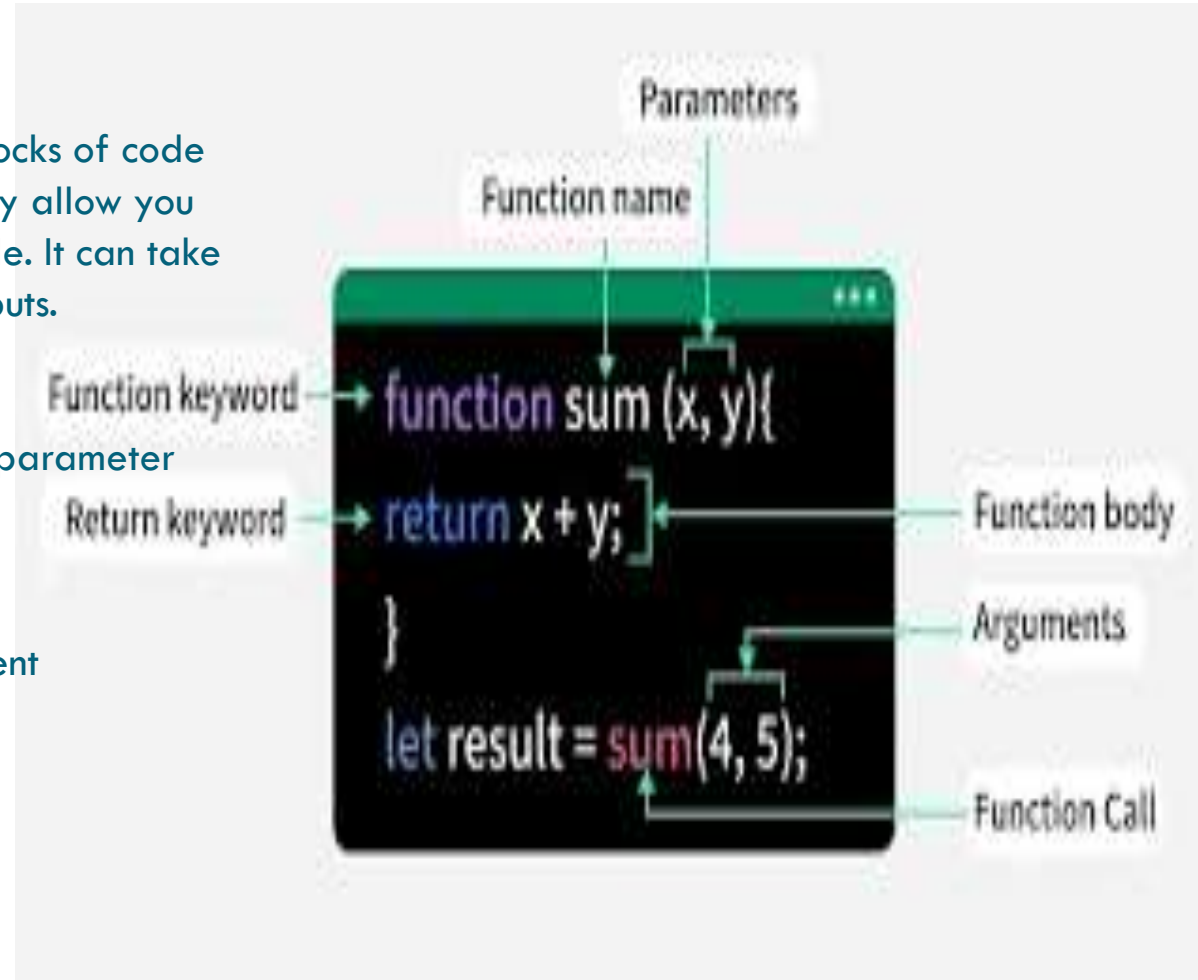
Functions in JavaScript

24

Functions in JavaScript are reusable blocks of code designed to perform specific tasks. They allow you to organize, reuse, and modularize code. It can take inputs, perform actions, and return outputs.

```
function greet(name) { // 'name' is a parameter
  console.log("Hello " + name);
}
```

```
greet("Alice"); // "Alice" is the argument
```




```
function name() {  
  statement ;  
  statement ;  
  ...  
  statement ;  
}
```

JS

```
function myFunction() {  
    alert("Hello!");  
    alert("How are you?");  
}
```

JS

- the above could be the contents of `example.js` linked to our HTML page
- statements placed into functions can be evaluated in response to user events

1. Named function:

A **Named function** is one that we write in code and then use whenever we need it by referencing its name and providing it with some parameters. Named functions come in handy when we need to call a function several times to give various values to it or run it multiple times.

```
function add(a, b) {  
  return a + b;}console.log(add(5, 4));
```

2. Anonymous function:

We can define a function in JavaScript without giving it a name. This nameless function is referred to as the **Anonymous function**. A variable must be assigned to an anonymous function.

```
let add = function (a, b) {  
  return a + b;}console.log(add(5, 4));
```

3. Nested Functions:

A function in JavaScript can contain one or more inner functions.

These **Nested Functions** fall within the purview of the outer function.

The inner function has access to the variables and arguments of the outer function.

However, variables declared within inner functions cannot be accessed by outer functions.

```
function addsquares(a,b)  
{  
  function square(x)  
  {  
    return x*x;  
  }  
  return square(a)+square(b);  
}  
document.write(addsquares(3,4));
```

4. Immediately invoked function expression (IIFE)

The browser executes the invoked function expression as soon as it detects it.

Immediately invoked function expression has the advantage of running instantly where it is situated in the code and producing direct output. That is, it is unaffected by code that occurs later in the script and can be beneficial.

```
let msg = (function() {  
  return "Welcome to mpec" ;})();console.log(msg);
```

5. Arrow Function (ES6)

A new way to write functions using the `=>` syntax. They are shorter and do not have their own this binding, which makes them useful in some cases.

```
const square = n => n * n;    FN_NAME=PARAMETERS=>RETUR  
console.log(square(4));
```

Callback Function

Definition: A function passed into another function as an argument, which is then invoked inside the outer function to complete some kind of routine or action.

Syntax: `functionOuter(functionCallback) ;`

```
1.function greet(name, callback) {  
  console.log("Hello " + name);  
  callback();  
}  
greet("Alice", () => console.log("Callback executed!"));
```

```
2.function num(n, callback) {  
  return callback(n);  
}  
const double = (n) => n * 2;  
console.log(num(5, double));
```

Constructor Function

Definition: A special function used to create and initialize objects. It is called with the `new` keyword. By convention, they start with a capital letter.

Syntax: `function Person(args) { this.prop = args; }`

```
function User(name) {  
  this.name = name;  
  this.isAdmin = false;  
}  
const user1 = new User("Bob");  
console.log(user1.name); // Bob
```

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}  
const user = new Person("Neha", 22);  
console.log(user.name);
```

Higher-Order Function

Definition: A function that either takes one or more functions as arguments or returns a function as its result.

Syntax:

```
const result = arr.map(fn);  
const numbers = [1, 2, 3];  
// .map is a higher-order function  
const doubled = numbers.map(num => num * 2);  
console.log(doubled); // [2, 4, 6]
```

Pure Function

Definition: A function that always returns the same output for the same input and has no "side effects" (it doesn't change variables outside its scope or modify global state).

Syntax:

```
const add = (a, b) => a + b;
```

// Pure: Predictable and clean

```
function add(a, b) {  
  return a + b;  
}
```

// Impure: Depends on external state

```
let tax = 5;  
function calculateTotal(price) {  
  return price + tax;  
}
```

Default Parameter Function

A function where parameters have default values if no argument is passed.
Helps avoid undefined issues.

```
function greet(name = "Guest") {  
  return "Hello, " + name; }  
console.log(greet()); console.log(greet("Aman"));
```

Rest Parameter Function

Definition: A syntax that allows a function to accept an indefinite number of arguments as an array.

Syntax: `function name(...args) {}`

```
function sumAll(...numbers) {  
  return numbers.reduce((acc, curr) => acc + curr, 0);  
}  
console.log(sumAll(1, 2, 3, 4, 5)); // 15
```

Type	Syntax Example	Key Characteristic
Named	<code>function name() { }</code>	Has a name; hoisted (can be called before defined).
Anonymous	<code>function() { }</code>	No name; usually assigned to variables or used as callbacks.
Expression	<code>const x = function() { }</code>	Function stored in a variable; not hoisted.
Arrow	<code>const x = () => { }</code>	Short syntax; no this binding.
IIFE	<code>((() => { })())</code>	Runs immediately after definition; creates local scope.
Callback	<code>fn(anotherFn)</code>	A function passed as an argument to another.
Constructor	<code>function User() { this.id = 1 }</code>	Used with new to create objects.

Type	Syntax Example	Key Characteristic
Async	<code>async function() { await ... }</code>	Handles promises; always returns a promise.
Generator	<code>function* gen() { yield 1 }</code>	Can pause/resume execution using <code>yield</code> .
Recursive	<code>fn() { if(...) return; fn() }</code>	A function that calls itself until a base case is met.
Higher-Order	<code>const x = (f) => f()</code>	Takes a function as input or returns one (e.g., <code>.map()</code>).
Nested	<code>function A() { function B() {} }</code>	Function inside a function; inner has access to outer's scope.
Pure	<code>(a, b) => a + b</code>	Same input = same output; no side effects.
Rest Param	<code>function(...args) {}</code>	Collects multiple arguments into a single array.

Exploring JavaScript Datatypes and Variables:

32

- ❑ `console.log(null === undefined)`
- ❑ `console.log(5 > 3 > 2)`
- ❑ `console.log([] === [])`-When you compare two arrays with `===`, you are comparing their references, not their contents.
- ❑ `console.log(true == 1)`
- ❑ `console.log(undefined > 0)`-When JavaScript attempts to compare undefined with 0, it converts undefined to NaN (Not-a-Number). Any comparison involving NaN returns false.
- ❑ `console.log("5" === 5)`-strict equality operator (`===`) checks both value and type
- ❑ `console.log([1, 2] == [1, 2])`
- ❑ `console.log(Infinity > 1000)`- Infinity represents an unbounded, positive number. It's greater than any finite number,

Adding JavaScript in HTML Document

33

- JavaScript can be included in an HTML document to add interactivity and dynamic behavior to web pages. It allows the browser to execute scripts that can modify content, handle events, and communicate with servers.
- **Inline JavaScript:** Code written directly inside HTML tags using attributes like onclick.
- **Internal JavaScript:** Script written inside the `<script>` tag within the HTML file.
- **External JavaScript:** JavaScript stored in a separate .js file and linked using the `<script src="file.js"></script>` tag.

Inline JavaScript

34

```
<html>
<head></head>
<body>
  <h2>
    Adding JavaScript in HTML Document
  </h2>
  <button onclick="alert('Button Clicked!')">
    Click Here
  </button>
</body>
</html>
```

External JavaScript (Using External File)

35

```
<html>
<head>
  <script src="script.js"></script>
</head>
<body>
  <h2>
    External JavaScript
  </h2>
  <h3 id="demo" style="color:green;">
    JAVATUTORIALS
  </h3>
  <button type="button" onclick="myFun()">
    Click Here
  </button>
</body>
</html>
```

How to Reference External JavaScript Files?

We can reference an external script in three ways in javascript:

36

- **By using a full URL:**

```
src = "https://www.JAVATUTORIAL.org/js/script.js"
```

- **By using a file path:**

```
src = "/js/script.js"
```

- **Without using any path:**

```
src = "script.js"
```

JavaScript Operators

JavaScript operators are symbols or keywords used to perform operations on values and variables. They are the building blocks of JavaScript expressions and can manipulate data in various ways.

37

Category	Operators
Arithmetic	<code>+, -, *, **, /, %, ++, --</code>
Assignment	<code>=, +=, -=, *=, /=, %=, **=, &=, `</code>
Comparison	<code>==, ===, !=, !==, >, <, >=, <=</code>
Logical	<code>&&, `</code>
Bitwise	<code>&, `</code>
Ternary	<code>condition ? exprIfTrue : exprIfFalse</code>
Type	<code>typeof, instanceof</code>
String	<code>+, +=</code>
Relational	<code>in, instanceof</code>

1. Arithmetic Operators

operator that operate upon the numerical values and return a numerical value.

38

- **Addition (\$+\$):** Adds two numbers or concatenates strings.
- **Subtraction (\$-\$):** Subtracts the right operand from the left operand.
- **Multiplication (\$*\$):** Multiplies two numbers.
- **Exponentiation (\$\$):** ****** Raises the first operand to the power of the second operand.
- **Division (\$/\$):** Divides the first operand by the second operand.
- **Modulus (\$\%\$):** Returns the integer remainder of a division.
- **Increment (\$++\$):** Increases an integer value by one.
- **Decrement (\$--\$):** Decreases an integer value by one.

```
// Number + Number => Addition  
let x = 1 + 2;  
console.log( x );
```

```
// Number + String => Concatenation  
let y = 5 + "hello";  
console.log( y );
```

```
// Number - Number => Subtraction  
let x = 10 - 7;  
console.log( x );
```

```
let y = "Hello" - 1;  
console.log( y );
```

```
// Number * Number => Multiplication  
let x = 3 * 3;  
let y = -4 * 4;  
console.log(x);  
console.log(y);
```

```
let a = Infinity * 0;  
let b = Infinity * Infinity;  
console.log(a);  
console.log(b);  
let z = 'hi' * 2;  
console.log(z);
```

2. Assignment Operators

assign values to variables. They can also perform operations like addition or multiplication while assigning the value.

39

- **Assignment (\$=\$):** Assigns the value of the right operand to the left operand.
- **Addition Assignment (\$+= \$):** Adds the right value to the variable and assigns the result to the variable.
- **Subtraction Assignment (\$-= \$):** Subtracts the right value from the variable and assigns the result.
- **Multiplication Assignment (\$*= \$):** Multiplies the variable by the right value and assigns the result.
- **Division Assignment (\$/= \$):** Divides the variable by the right value and assigns the result.
- **Remainder Assignment (\$%= \$):** Divides the variable by the right value and assigns the remainder.
- **Exponentiation Assignment (\$= \$):**** Raises the variable to the power of the right operand and assigns the result.

```
let n = 10;  
n += 5;  
n *= 2;  
console.log(n);
```

```
let a = 50;  
console.log( a %= 10);
```

```
let a = 2;  
console.log(a **= 3);
```

```
let a = 10;  
const b = 2;  
console.log(a /= b);
```

3. Comparison Operators

Comparison operators compare two values and return a boolean (true or false). They are useful for making decisions in conditional statements.

40

- **Equal (\$==\$):** Returns true if operands are equal after type conversion.
- **Strict Equal (\$===\$):** Returns true only if operands are equal and of the same type.
- **Not Equal (\$!=\$):** Returns true if operands are not equal, regardless of type.
- **Strict Not Equal (\$!==\$):** Returns true if operands are not equal or not the same type.
- **Greater Than (\$>\$):** Returns true if the left value is larger than the right.
- **Greater Than or Equal (\$>=\$):** Returns true if the left value is larger than or equal to the right.
- **Less Than (\$<\$):** Returns true if the left value is smaller than the right.
- **Less Than or Equal (\$<=\$):** Returns true if the left value is smaller than or equal to the right.

```
console.log(10 > 5);  
console.log(10 === "10");  
let x = 5;  
let y = '5';  
// Checking of operands  
console.log(x !== 6);  
console.log(y !== '5');  
let x = 5;  
let y = '5';  
// Checking of operands  
console.log(x === 6);  
console.log(y === '5');
```


4.Logical Operators

combine or modify boolean values to make decisions. They help control program flow by evaluating whether one or more conditions are true or false.

41

□ 1. Logical AND (&&) Operator

- The logical AND (&&) operator checks whether both operands are true. If both are true, the result is true. If any one or both operands are false, the result is false.

```
// Check if both conditions are true
```

```
let age = 20;
```

```
let idProof = true;
```

```
// Logical AND checks both conditions
```

```
if (age >= 18 && idProof) {
```

```
    console.log("Allowed");
```

```
} else {
```

```
    console.log("Not Allowed");
```

```
}
```

MPEC2026

It treats false, 0, -0, "", null, undefined, NaN and document.all as false.

Logical OR (||) Operator

operator checks whether at least one of the operands is true. If either operand is true, the result is true. If both operands are false, the result is false.

42

```
// Check if at least one condition is true
let age = 16;
let hasGuardian = true;
// Logical OR checks if either condition is true
if (age >= 18 || hasGuardian) {
  console.log("Allowed");
} else {
  console.log("Not Allowed");
}
```

Logical NOT (!) Operator

inverts the boolean value of its operand.

```
let isLoggedIn = false;

if (!isLoggedIn) {
  console.log("Log in!");
} else {
  console.log("Welcome back!");
}
```

Type & Special Operators

43

- typeof**: Returns a string indicating the data type of the operand.
- instanceof**: Returns true if an object is an instance of a specific constructor.
- Ternary (\$? :\$)**: A shorthand for if-else that returns one of two values based on a condition.
- Nullish Coalescing (\$??)**: Returns the right-hand side when the left-hand side is null or undefined.
- Optional Chaining (\$?.)**: Returns undefined instead of throwing an error if a nested property is missing.

Operator	Description	Code Example
typeof	Checks the data type.	<code>console.log(typeof 42); // "number"</code>
instanceof	Checks if an object belongs to a class.	<code>[1, 2] instanceof Array; // true</code>
? : (Ternary)	Short if-else statement.	<code>let status = (age >= 18) ? "Adult" : "Minor";</code>
?? (Nullish)	Provides a fallback for null/undefined.	<code>let name = username ?? "Anonymous";</code>
?. (Optional Chaining)	Reads nested properties safely.	<code>let city = user?.address?.city; // undefined (no crash)</code>
delete	Removes a property from an object.	<code>delete user.password;</code>
in	Checks if a property exists in an object.	<code>"name" in user; // true</code>
void MPEC2026	Evaluates expression and returns undefined.	<code>Click</code>

```
<div class="calculator">
  <h2>Simple Calc</h2>

  <input type="number" id="num1" placeholder="Enter first number">
  <input type="number" id="num2" placeholder="Enter second number">

  <div class="buttons">
    <button onclick="calculate('sum')">Sum</button>
    <button onclick="calculate('minus')">Minus</button>
    <button onclick="calculate('multiply')">Multiply</button>
  </div>

  <h3 id="result">Result: 0</h3>
</div>

/* Styling the Input Fields */
input {
  width: 200px;
  height: 40px;
  margin: 10px;
  display: block;
  border: 1px solid #ccc;
  padding: 5px;
```

```
/* Styling the Buttons */  
button {  
    width: 80px;  
    height: 40px;  
    cursor: pointer;  
    background-color: #eee;  
    border: 1px solid #999;  
    margin: 5px;  
    transition: 0.3s; /* Makes the color change smooth */  
}
```

```
/* The Hover Property */  
button:hover {  
    background-color: pink;  
}
```

```
/* Centering the calculator slightly */  
.calculator {  
    padding: 20px;  
    font-family: Arial, sans-serif;  
}
```

Control Statements in JavaScript

JavaScript **control statement** is used to control the execution of a program based on a specific condition. If the condition meets then a particular block of action will be executed otherwise it will execute another block of action that satisfies that particular condition.

Types of Control Statements in JavaScript

Conditional Statement - These statements are used for decision-making, a decision

Iterative Statement- This is a statement that iterates repeatedly until a condition is met.

If Statement

In this approach, we are using an if statement to check a specific condition, the code block gets executed when the given condition is satisfied.

Syntax

```
if (condition) {  
    // code to execute if the condition is true  
}
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>If Statement Example</title>
```

```
</head>
```

```
<body>
```

```
<h2>If Statement Example</h2>
```

```
<script>
```

```
let age = 20;
```

```
if (age >= 18) {
```

```
    document.write("You are eligible to vote");
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

else Statement

else to specify a code block to be executed, if the same condition is false.

```
if (condition) {  
    // code to execute if the condition is true  
} else {  
    // code to execute if the condition is false  
}
```

```
<!DOCTYPE html>  
<html>  
<head>  
<title>If Else Example</title>  
</head>  
<body>  
<h2>If Else Statement Example</h2>  
<script>  
let number = 7;  
if (number % 2 == 0) {  
    document.write("Number is Even");  
}  
else {  
    document.write("Number is Odd");  
}  
</script>  
</body>  
</html>
```

else if Statement

Use else if to specify a **new condition** to test, if the first condition is false.

Syntax

```
if (condition1) {  
    // code to execute if condition1 is true  
} else if (condition2) {  
    // code to execute if the condition1 is false and condition2 is true  
} else {  
    // code to execute if the condition1 is false and condition2 is false  
}
```

```
<!DOCTYPE html>  
<html>  
<head>  
<title>Else If Example</title>  
</head>  
<body>  
<h2>Else If Statement Example</h2>  
<script>  
  
let marks = 75;  
if (marks >= 90) {  
    document.write("Grade A");  
}
```

```

}
else if (marks >= 60) {
    document.write("Grade B");
}
else {
    document.write("Grade C");
}
</script>
</body>
</html>

```

switch Statement

Syntax-

```

switch (expression) {
    case value1:
        statement1;
        break;
    case value2:
        statement2;
        break;
    case valueN:
        statementN;
        break;
    default:
        statementDefault;
}

```

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>JavaScript Control Flow</h1>
```

```
<h2>The switch Statement</h2>
```

```
<p id="demo"></p>
```

```
<script>
```

```
let day;
```

```
let date = new Date().getDay();
```

```
switch (date) {
```

```
    case 0:
```

```
        day = "Sunday";
```



```

        break;
    case 1:
        day = "Monday";
        break;
    case 2:
        day = "Tuesday";
        break;
    case 3:
        day = "Wednesday";
        break;
    case 4:
        day = "Thursday";
        break;
    case 5:
        day = "Friday";
        break;
    case 6:
        day = "Saturday";
    }
    document.getElementById("demo").innerHTML = "Today is " + day;
</script>
</body>
</html>

```

NOTE- The `getDay()` method returns the weekday as a number between 0 and 6.
 (Sunday=0, Monday=1, Tuesday=2 ..)

Ternary Operator (? :)

Use (? :) (ternary) as a **shorthand** for if...else.

Example

condition ? expression1 : expression2

condition ? value if true : value if false

EX-

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Ternary Operator</h1>
<p id="demo"></p>
<script>
let age = 16;
let text = (age < 18) ? "Minor" : "Adult";
document.getElementById("demo").innerHTML = text;
</script>
</body>
</html>
```

Using [For loop](#)

In this approach, we are using for loop in which the execution of a set of instructions repeatedly until some condition evaluates and becomes false

Syntax

```
for (statement 1; statement 2; statement 3) {
  // Code here . . .
}
```

Now let's understand this with the help of example:

```
<!DOCTYPE html>
<html>
<head>
<title>For Loop Example</title>
</head>
<body>
<h2>For Loop Example (Even Numbers)</h2>
<script>

for (let i = 0; i <= 10; i++) {

  if (i % 2 === 0) {
    document.write(i + "<br>");
```

```
}  
  
}  
  
</script>  
  
</body>  
</html>
```

Output

```
0  
2  
4  
6  
8  
10
```

Using While loop

The while loop repeats a block of code as long as a specified condition is true.

Syntax

```
while (condition) {  
    // code block  
}
```

Now let's understand this with the help of example

```
<!DOCTYPE html>  
<html>  
<head>  
<title>While Loop Example</title>  
</head>  
<body>  
<h2>While Loop Example</h2>  
<script>  
  
let i = 1;  
  
while (i <= 5) {  
    document.write(i + "<br>");  
    i++;  
}  
  
</script>  
</body>  
</html>
```

Output

1
2
3
4
5

Using Do-While loop

The do-while loop is similar to the while loop, except that the condition is evaluated after the execution of the loop's body. This means the code block will execute at least once, even if the condition is false.

Syntax

```
do {  
    // code block  
} while (condition);
```

Now let's understand this with the help of example:

```
<!DOCTYPE html>  
<html>  
<head>  
<title>Do While Loop Example</title>  
</head>  
<body>  
<h2>Do While Loop Example</h2>  
<script>  
let i = 1;  
  
do {  
    document.write(i + "<br>");  
    i++;  
}  
while (i <= 5);  
</script>  
</body>  
</html>
```

Output

1
2
3
4

break Statement

```
<!DOCTYPE html>  
<html>  
<head>
```

```
<title>Break Example</title>
</head>
<body>
<h2>Break Statement Example</h2>
```

```
<script>
for (let i = 1; i <= 10; i++) {

  if (i == 5) {
    break;
  }

  document.write(i + "<br>");

}
```

```
</script>
```

```
</body>
</html>
```

continue Statement

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Continue Example</title>
```

```
</head>
```

```
<body>
```

```
<h2>Continue Statement Example</h2>
```

```
<script>
```

```
for (let i = 1; i <= 5; i++) {
```

```
  if (i == 3) {
```

```
    continue;
```

```
  }
```

```
  document.write(i + "<br>");
```

```
}
```

```
</script>
```

</body>

</html>